

Haifa K-series Pipeline for HPC, IPC, and the Haifa Cluster

This report summarizes how we turned raw financial data for **Haifa Port Company (HPC, legacy operator)** and the **Israel Ports Company's Haifa assets (IPC, landlord / infrastructure owner)** into **real, operator-level and cluster-level monthly capital stock series**. It is written as a handoff for future work (and future chats) so that the full logic of the pipeline is transparent and reproducible.

The report is organized as follows:

1. Big-picture role of K in the thesis
2. Data sources and overall design
3. HPC Track B pipeline: from raw financials to monthly Path B K
4. IPC Haifa pipeline: from raw IPC tables to monthly PIM-lin K
5. Haifa cluster K: combining HPC Path B and IPC PIM-lin
6. Key choices, QA, and interpretation for the thesis
7. Remaining extensions and to-dos

1. Big-picture role of K in the thesis

The thesis studies how Israel's port-sector reforms affected **labor productivity, capital deepening, and throughput**. To estimate dynamic treatment effects credibly, you need measures of capital that are:

- **Real** (deflated) and comparable across years
- **Operator- and owner-specific** (HPC vs IPC, and later SIPG / APC / TIL)
- Available at least **annually**, preferably **monthly**, so that they can line up with monthly throughput and labor data

You also need K for two distinct econometric roles:

1. As an **outcome / mediator**, e.g. $\ln(K/L)$ or $\ln(K)$ in a mediation-style event study (Model 1B).
2. As a **control** in throughput regressions (Model 2), to separate demand/traffic shocks from capacity changes.

The K pipeline therefore has to be **transparent, traceable to financial statements**, and flexible enough to support alternative depreciation assumptions and robustness checks.

2. Data sources and overall design

2.1 Core inputs

For Haifa you now have two distinct capital owners:

- **HPC (Haifa Port Company, legacy operator)** – cranes, yard equipment, some buildings and vehicles
- **IPC (Israel Ports Company, Haifa cluster)** – berths, breakwaters, marine works, landside infrastructure, land and buildings

The core inputs are:

- **HPC raw financials** – `haifa_financials_raw.tsv`
- **IPC raw tables** – `haifa_ipc_raw_table1.csv` and `haifa_ipc_raw_table2.csv`
- **OECD capital deflator** – quarterly non-residential GFCF price index for Israel, used for both HPC and IPC (rebased to 2019 = 1).
- **Haifa big projects table** – `Haifa_big_projects.csv` with a curated list of large HPC capex items (cranes, tugboats, berth works, etc.).

All monetary data are in **thousands of NIS**, historical cost, under IFRS.

2.2 Shared architectural choices

Across both companies, the pipeline uses the same three conceptual steps:

1. **Step 1: Raw → real annual financials**
2. Standardize and clean the financial tables.
3. Merge in the capital deflator.
4. Compute **real net PPE**, **real investment**, **real depreciation**, and (where available) **real disposals**.
5. Save a JSON file specifying a **set of depreciation scenarios** (`low`, `central`, `high`).

6. Step 2: Annual PIM backbone (Track B)

7. Use a **Perpetual Inventory Method (PIM)** to generate annual capital stocks under each depreciation scenario.
8. Anchor the PIM at the first year's observed real net PPE, then recurse forward:

$$K_t = (1 - \delta)K_{t-1} + I_t - D_t.$$

9. Produce annual stocks `K_PIM_real_low/central/high`, plus QA gaps vs the accounting net PPE.

10. Step 3: Monthly K series

11. For HPC: two monthly paths
 - **Path A** – smooth PIM-based K via linear time interpolation between December anchors.
 - **Path B** – lumpy K that explicitly incorporates big projects.
12. For IPC: **Path A only** (PIM + linear), since you do not yet have a curated big-projects list for IPC.

Finally, you combine **HPC Path B** with **IPC PIM-lin** to get a **Haifa cluster K series**.

The rest of the report walks through these steps in more detail.

3. HPC Track B pipeline: from raw financials to monthly Path B K

3.1 Step 1 – Real-valued HPC financials (00_build_K_step1.py)

Goal. Take the raw Haifa Port Company financial extraction and produce a clean annual panel of **real** wealth and flow variables suitable for PIM.

Inputs

- `haifa_financials_raw.tsv` – hand-constructed from HPC’s annual reports.
- OECD QNA capital deflator CSV – quarterly non-residential GFCF index for Israel.

Key operations

1. **Standardize core columns.**

2. Map verbose raw column names to internal names:

- `ppe_net_nominal_thousands_nis_eoy` → `ppe_net_nom` (year-end net PPE, nominal)
- `purchase_of_fixed_assets_thousands_nis_cashflow` → `additions_nom` (cash-flow capex)
- `depr_amort_total_thousands_nis_cf_adj` → `depr_nom` (depreciation + amortization)
- `proceeds_from_realization_fixed_assets_thousands_nis` → `disposals_nom` (disposals from cash-flow)

3. Clean the numeric fields (remove commas, spaces) and coerce to floats.

4. **Parse the deflator.**

5. Read the OECD file, extract the numeric index column, and parse `period` strings (e.g. `1995-Q1`) to **years**.

6. Compute the **annual average index** for each year and rebase so that **2019 = 1.0**.

7. Result: a simple `[year, deflator]` table covering at least 1995–2025.

8. **Attach real values.**

9. Merge the financials with the annual deflator on `year`.

10. Compute real variables:

- `K_book_real = ppe_net_nom / deflator` – book net PPE in real terms.
- `I_real = additions_nom / deflator` – real annual cash-flow investment.
- `depr_real = depr_nom / deflator`.

- `disposals_real = disposals_nom / deflator` (where available).

11. Depreciation scenarios metadata.

12. Define a dictionary of annual depreciation rates:

- `low = 0.04`, `central = 0.06`, `high = 0.08`.

13. Write `00_depreciation_choice.json` recording the chosen scenario (`central`) and the set of available scenarios.

Outputs

- `00_haifa_financials_step1_real.tsv` – one row per year with real wealth/flow variables and provenance fields.
- `00_haifa_financials_step1_sample.csv` – first 10 rows for quick inspection.
- `00_depreciation_choice.json` – depreciation scenarios metadata.

This step ensures that all subsequent K series are in **real thousands of NIS (2019 prices)**.

3.2 Step 2 – Annual PIM backbone (`01_build_K_annual_PIM.py`)

Goal. Construct an annual PIM-based capital stock series for HPC under each depreciation scenario, while retaining a tight connection to the accounting net PPE.

Inputs

- `00_haifa_financials_step1_real.tsv` (Step 1 output).
- `00_depreciation_choice.json` (depreciation scenarios).

Key operations

1. **Load real financials.**

2. Keep core variables:

- `company`, `year`, `K_book_real`, `I_real`, `depr_real`, `disposals_real`, `deflator`, some nominal fields and provenance.

3. **Define PIM investment** `I_real_pim`.

4. Start from `I_real` (real cash-flow investment, signed).

5. For PIM, what matters is **gross acquisitions**, so the script uses:

- `I_real_cf_signed = I_real` (for reference), and
- `I_real_pim = abs(I_real)` (non-negative PIM investment series).

6. There is a toggle to include real intangibles in PIM, but you leave this **off**, so intangibles are excluded from Track B.

7. **PIM recursion by company and scenario.**

8. For each company (here: just HPC) and each depreciation rate δ in {low, central, high}:
 - Sort the years and **drop rows without** K_book_real (scaffolding or incomplete rows).
 - Take the first year as an anchor: set K_PIM_real_scen = K_book_real for that year.
 - For each subsequent year, apply:
 - Deal with multi-year gaps by iterating depreciation with **zero flows** in missing years.
 - Then apply the PIM formula:

$$K_t^{\text{PIM}} = (1 - \delta)^{\text{gap}} K_{t-1}^{\text{PIM}} + I_t - D_t.$$

- Record for each year:
- K_PIM_real_low/central/high
- Gap vs book: gap_book_minus_PIM_low/central/high = K_book_real - K_PIM_real_scen.
- flows_imputed_flag - indicates if flows were inferred (e.g. missing disposals or multi-year gaps).
- gap_years_from_prev - number of years since the previous observation.

Outputs

- 01_K_B_annual_Haifa_PIM.tsv - annual PIM table with stocks and QA.
- 01_K_B_annual_Haifa_PIM_sample.csv - first 10 rows.
- 01_PIM_config.json - metadata about scenarios, inputs, and toggles.

This file is the **annual backbone** for both the smooth monthly series and the Path B series.

3.3 Step 3 – Monthly smooth PIM series (Path A, 02_build_K_monthly_PIM_linear.py)

Goal. Convert the annual PIM backbone into a **monthly** K series by interpolating between year-end stocks.

Inputs

- 01_K_B_annual_Haifa_PIM.tsv (annual backbone).

Key operations

1. **Identify PIM stock columns.**
2. Automatically detect all K_PIM_real_* columns, e.g. low, central, high.
3. **Build December anchors and monthly skeleton.**
4. For each company:
 - Create an anchor row for each year with date **1 December** of that year (YYYY-12-01) and the corresponding K_PIM_real_* values.
 - Construct a monthly date range from the **first** December anchor to the **last**, with frequency MS (month-start).

- Merge anchor values into this monthly skeleton, leaving non-December months with `NaN` in the anchor columns.

5. Interpolate K in levels between December anchors.

6. For each scenario, create `K_PIM_lin_scen` as a **time interpolation** of the anchor series.
7. This yields a smooth monthly K path that exactly matches the PIM stocks every December.

8. Propagate flags and labels.

9. Map `flows_imputed_flag` and `gap_years_from_prev` from the annual table down to months by calendar year:
 - `flows_imputed_annual` – same for all months in a given year.
 - `gap_years_from_prev_annual` – same for all months in a given year.
10. Add identifiers and structural flags:
 - `port = "Haifa"`
 - `company = "Haifa Port Company (legacy)"`
 - `operator_or_owner = "Haifa Port Company (legacy)"`
 - `imputed_2022 = 1` for all months in 2022 (you treat 2022 as structurally imputed because of the privatization transition and limited direct anchors).

Outputs

- `02_K_B_monthly_Haifa_PIM_lin.tsv` – smooth monthly Path A K series for HPC.
- `02_K_B_monthly_Haifa_PIM_lin_sample.csv` – first 24 rows.
- `02_PIM_lin_config.json` – metadata on interpolation method, flags, etc.

Path A is a **clean, model-based K series**; you ultimately use it as the **background** in Path B.

3.4 Step 4 – Monthly lumpy series (Path B, `03_build_K_monthly_PathB_lumpy.py`)

Goal. Build a **lumpy monthly K series** that explicitly incorporates major HPC projects, while preserving consistency with the annual PIM backbone.

Inputs

- Annual PIM backbone (`01_K_B_annual_Haifa_PIM.tsv`).
- Monthly skeleton with dates and flags (`02_K_B_monthly_Haifa_PIM_lin.tsv`).
- Big-projects list (`Haifa_big_projects.csv`).
- Real financials (`00_haifa_financials_step1_real.tsv`) to obtain the deflator by year.

Key operations

1. **Load annual PIM and monthly skeleton.**
2. Restrict the annual PIM table to `company = "Haifa Port Company (legacy)"`.
3. Use the monthly skeleton **only for dates and flags**; the Path A K levels are not used directly.

4. Build deflator-by-year map.

5. From the Step 1 real financials, construct a dictionary `year → deflator` (taking the first deflator value per year).

6. This ensures that big-project costs are deflated using the **same price index** as the main PIM model.

7. Load and clean the big-projects table.

8. Filter to `port = Haifa` and then, where possible, to projects whose operator/owner field matches HPC (e.g. containing "Haifa Port Company" or "HPC").

9. Filter to projects flagged as relevant for **Path B** (e.g. `relevant_for_K_B = 1`), if such a column exists; otherwise keep all Haifa HPC projects.

10. Parse an **investment year**:

- Prefer an explicit `investment_year` or `investment_year_fs` field (extract the first four-digit year, e.g. `2021-2025` → 2021).
- Fall back to `start_year`, `expected_end_year`, or `commissioning_year` if needed.

11. Infer a **commissioning month** `commissioning_month` as a `YYYY-MM-01` timestamp using a cascade of fields:

1. `commissioning_year` + `commissioning_month` (if available)
2. `expected_end_year` + `expected_end_month` (default month = 12)
3. `start_year` + `start_month` (default month = 6)
4. `investment_year` (default month = 12)

12. Deflate nominal project costs to real thousands of NIS:

- If a real-cost column already exists, use it.
- Otherwise, divide `amount_nominal_th_nis` (or similar) by the appropriate year's deflator (with nearest-year fallback) to get `cost_real_thousands_nis`.

13. Ensure every project has an **asset life**:

- Use `asset_life_years` if present.
- Otherwise, set to a default of **28 years**, consistent with an aggregate PPE life.

14. Build per-project capital paths.

15. For each project and for a common monthly date index:

- Before the commissioning month, $K = 0$.
- At the commissioning month, K jumps to the full real cost.
- Thereafter, the project depreciates geometrically at a monthly rate derived from the annual rule:
 - $\text{delta_annual} = 1 / \text{asset_life_years}$.
 - $\text{delta_month} = 1 - (1 - \text{delta_annual})^{**} (1 / 12)$.
- This yields a series `K_proj_{id}(t)` for each project.

16. Aggregate big projects to `K_big_central,t`.

17. Sum all per-project capital levels at each month to get `K_big_central`.
18. Count, per month, how many projects commission:
 - `num_projects_commissioned` – number of projects starting that month.
 - `lumpy_event_month = 1` if any project starts, else 0.
19. **Decompose annual PIM into big and background K.**
20. For each year in the annual PIM backbone:
 - Take `K_big_y = K_big_central` at **December** of that year.
 - Compute the implied **background stock**:
 - `K_bg_y = K_PIM_real_central_y - K_big_y`.
 - If `K_bg_y` is negative (i.e. big projects alone exceed the PIM stock), clamp to a tiny positive epsilon to avoid $\log(0)$ issues.
21. Store `K_bg_y`, along with the PIM flags `flows_imputed_flag` and `gap_years_from_prev`.
22. **Build monthly background K.**
23. Treat the annual `K_bg_y` as anchors at each December.
24. Log-linear interpolate between anchors:
 - Create a monthly skeleton over the same date range.
 - Merge `K_bg_y` into December months as `K_bg_anchor`.
 - Interpolate `$\log(K\_bg\_anchor)$` over time and exponentiate to get `K_bg_central`.
25. **Combine big and background into Path B monthly K.**
26. Use the original monthly skeleton from Step 3 for dates and flags.
27. Merge in `K_big_central`, `lumpy_event_month`, `num_projects_commissioned` and `K_bg_central`.
28. Compute total Path B K:
 - `K_PathB_central = K_big_central + K_bg_central`.
29. Ensure `K_PIM_real_central` is available at December anchors, either carried from the skeleton or re-merged from the annual PIM table.
30. Compute a QA gap at anchors:
 - `gap_PathB_minus_PIM_central = K_PathB_central - K_PIM_real_central`.

Outputs

- `03_K_B_monthly_Haifa_PathB.tsv` – main HPC series used downstream.
- `03_K_B_monthly_Haifa_PathB_sample.csv` – first 24 rows.
- `03_K_B_PathB_QA.tsv` – Gap table at annual anchors, for QA.
- `03_PathB_config.json` – config and provenance.

Interpretation. Path B is your preferred **HPC operator-level K series**. It:

- Respects accounting net PPE dynamics (through reconciliation with PIM).
- Explicitly models big projects as **lumpy jumps**, so that months with commissioning events will show visible changes in K.
- Provides a smooth background component capturing all the other, smaller capex and depreciation.

4. IPC Haifa pipeline: raw IPC tables → monthly PIM-lin K

Unlike HPC, for IPC you currently have no big-projects table; all large civil works for Bayport and Haifa deepening are embedded in IPC's aggregated capex. For now, you therefore build a **single Path A series** for IPC: PIM + linear monthly interpolation.

4.1 IPC raw data: Table 1 and Table 2

You extracted IPC data for Haifa from IPC's statistical / financial annexes into two CSVs:

1. `haifa_ipc_raw_table1.csv` – **PPE by asset class**
2. For each year 2018–2024 and asset class (total, WIP, marine, buildings, land, etc.), includes:
 - `cost_open`, `additions`, `transfers`, `cap_interest`, `disposals`, `cost_close`.
 - `accdep_open`, `depreciation_expense`, `accdep_disposals`, `accdep_close`.
 - `net_ppe_close` (closing net PPE).
3. Some asset-class level depreciation fields are missing for later years; the **total row** is generally complete.
4. `haifa_ipc_raw_table2.csv` – **cash-flow based flows**
5. For each year, contains:
 - `dep_amort_th_nis`
 - `purchase_ppe_cf_th_nis` (negative cash outflow)
 - `capitalized_interest_ppe_th_nis` (negative)
 - `proceeds_sale_ppe_cf_th_nis` (positive inflow)
6. You construct an approximate **gross investment** as:

$$\text{approx_gross_ppe_investment} = -\text{purchase_ppe_cf} - \text{cap_interest} + \text{proceeds_sale_ppe}.$$

Together, these tables give you **net stocks** and **flow measures** that are conceptually aligned with the HPC extraction.

4.2 Step 10 – IPC Step 1: real-valued IPC financials (`10_build_IPC_step1_real.py`)

Goal. Mirror the HPC Step 1 logic for IPC, producing a single annual real-valued panel for Haifa IPC.

Inputs

- `haifa_ipc_raw_table1.csv`
- `haifa_ipc_raw_table2.csv`
- OECD QNA deflator file (same as for HPC)

Key operations

1. Load and harmonize IPC tables.

2. Read Table 1 and Table 2 into DataFrames.

3. For Table 1:

- Keep the **total** row per year, as it has the most complete net PPE fields.
- Set `ppe_net_nom = net_ppe_close`.

4. For Table 2:

- Set `additions_nom` equal to `approx_gross_ppe_investment_th_nis` (constructed as above).
- Set `depr_nom = depr_amort_th_nis`.
- Set `disposals_nom` based on `proceeds_sale_ppe_cf_th_nis` (or zero if you choose to treat disposals implicitly).

5. Merge the two tables by `year` and attach a company label:

- `company = "Israel Ports Company (Haifa cluster)"`.

6. Attach the deflator and compute real variables.

7. Use the same `load_deflator` function as for HPC to parse the OECD file and construct an annual `[year, deflator]` table (2019=1).

8. Merge with the combined IPC table and compute:

- `K_book_real = ppe_net_nom / deflator`.
- `I_real = additions_nom / deflator`.
- `depr_real = depr_nom / deflator`.
- `disposals_real = disposals_nom / deflator` (where relevant).

9. Depreciation scenarios metadata.

10. Create `10_IPC_depreciation_choice.json` with the same scenario set as HPC:

- `low = 0.04`, `central = 0.06`, `high = 0.08`.

Outputs

- `10_haifa_IPC_financials_step1_real.tsv` – annual real wealth/flow panel for IPC.
- `10_haifa_IPC_financials_step1_sample.csv` – first 10 rows.
- `10_IPC_depreciation_choice.json` – IPC depreciation scenarios.

4.3 Step 11 – Annual IPC PIM backbone (11_build_K_annual_IPC_PIM.py)

Goal. Apply the exact same PIM logic as for HPC to the IPC real financials, generating annual capital stocks under each depreciation scenario.

Inputs

- 10_haifa_IPC_financials_step1_real.tsv.
- 10_IPC_depreciation_choice.json.

Key operations

The code is essentially a **copy of the HPC** 01_build_K_annual_PIM.py, with file paths changed:

- Use `I_real_pim = abs(I_real)` as the gross investment series.
- Anchor PIM at the first year's `K_book_real`.
- Apply the PIM recursion for each scenario δ in {4%, 6%, 8%}.
- Track `flows_imputed_flag` and `gap_years_from_prev` (in practice there are no multi-year gaps in 2018–2024, so gaps are 1 and flags are `False`).

Outputs

- 11_K_B_annual_Haifa_IPC_PIM.tsv – annual IPC PIM table.
- 11_K_B_annual_Haifa_IPC_PIM_sample.csv – sample.
- 11_IPC_PIM_meta.json – metadata.

4.4 Step 12 – Monthly IPC PIM-lin K (12_build_K_monthly_IPC_PIM_linear.py)

Goal. Build a monthly PIM-lin capital series for IPC, parallel to HPC's Path A.

Inputs

- 11_K_B_annual_Haifa_IPC_PIM.tsv.

Key operations

The logic mirrors 02_build_K_monthly_PIM_linear.py:

1. **Load annual IPC PIM table** and detect `K_PIM_real_*` columns.
2. **Build anchors and monthly skeleton.**
3. Create December anchors with `K_PIM_real_low/central/high` at `YYYY-12-01`.
4. Construct a monthly date range from the first to the last December anchor.
5. **Interpolate monthly PIM-lin K.**

6. For each scenario, create `K_PIM_lin_scen` by time interpolation.

7. Add identifiers and flags.

8. `port = "Haifa"`.
9. `company = "Israel Ports Company (Haifa cluster)"`.
10. `operator_or_owner = "Israel Ports Company (landlord / infrastructure owner)"`.
11. Propagate annual `flows_imputed_flag` and `gap_years_from_prev` to monthly flags (`flows_imputed_annual`, `gap_years_from_prev_annual`).
12. There is no special 2022 imputation rule for IPC; however, when combining with HPC, you later drop months where both series are unreliable.

Outputs

- `12_K_B_monthly_Haifa_IPC_PIM_lin.tsv` - monthly IPC K.
- `12_K_B_monthly_Haifa_IPC_PIM_lin_sample.csv` - sample.
- `12_IPC_PIM_lin_meta.json` - metadata.

This series is your current best measure of **IPC's Haifa capital stock** at monthly frequency.

5. Haifa cluster K: combining HPC Path B and IPC PIM-lin (`20_build_Haifa_cluster_K_merge.py`)

5.1 Goal and design

The goal of the cluster step is to produce a **single panel** that contains:

1. **HPC operator-level K** (Path B),
2. **IPC Haifa-level K** (PIM-lin), and
3. Their **sum**, representing the **Haifa port cluster's total K**.

This panel is designed to be easy to **merge into your main terminal × month panel** and to support both operator-level and cluster-level regressions.

5.2 Inputs

- `03_K_B_monthly_Haifa_PathB.tsv` - HPC Path B K.
- `12_K_B_monthly_Haifa_IPC_PIM_lin.tsv` - IPC PIM-lin K.

5.3 Merge logic

1. **Load HPC Path B and select the main K column.**
2. Filter `03_K_B_monthly_Haifa_PathB.tsv` to `company = "Haifa Port Company (legacy)"`.
3. Keep core fields:
 - `port`, `company`, `operator_or_owner`, `month`, `year`, `K_PathB_central`.

4. Rename `K_PathB_central` → `K_central` in an intermediate DataFrame.
5. **Load IPC monthly PIM-lin and select the main K column.**
6. Filter `12_K_B_monthly_Haifa_IPC_PIM_lin.tsv` to `company = "Israel Ports Company (Haifa cluster)"`.
7. Keep:
 - `port`, `company`, `operator_or_owner`, `month`, `year`, `K_PIM_lin_central`.
8. Rename `K_PIM_lin_central` → `K_central`.
9. **Merge on** `(port, month, year)`.
10. Perform an **outer join** between the HPC and IPC frames on `port`, `month`, `year`.
11. Rename merged columns:
 - HPC central K → `K_HPC_central`.
 - IPC central K → `K_IPC_central`.
12. **Drop unreliable early months.**
13. You decided that neither the HPC nor IPC series provide reliable month-level information for **Jan-Nov 2018** (HPC is only anchored at Dec 2018; IPC's early months are purely interpolated from that anchor).
14. The merge script therefore explicitly **drops all rows with** `year == 2018` **and** `month < 2018-12-01`.
15. The Haifa K panel thus starts in **December 2018**, which is also when your main throughput panel becomes reliable.
16. **Compute cluster K.**
17. Define the cluster's total capital stock as a row-wise sum:
 - `K_cluster_central = K_HPC_central + K_IPC_central`, using `min_count=1` so that if one component is missing but the other is present, the sum equals the available component.
18. In practice, after dropping early 2018 rows, both components are present for every month; the cluster K series is therefore a simple sum.
19. **Construct a long-format panel with three company values.**

20. Starting from the merged monthly table, construct three stacked DataFrames:

1. HPC rows

- `company = "Haifa Port Company (legacy)".`
- `operator_or_owner = "Haifa Port Company (legacy operator)".`
- `K_central = K_HPC_central.`

2. IPC rows

- `company = "Israel Ports Company (Haifa cluster)".`
- `operator_or_owner = "Israel Ports Company (landlord / infrastructure owner)".`
- `K_central = K_IPC_central.`

3. Cluster rows

- `company = "Haifa cluster (HPC + IPC)".`
- `operator_or_owner = "Haifa port cluster (HPC + IPC)".`
- `K_central = K_cluster_central.`

21. Concatenate these three DataFrames vertically to get a **long panel** with one row per (port, company, month) combination.

22. **Column ordering and save.**

23. Columns in the final file:

- port
- company
- operator_or_owner
- month
- year
- K_central - the relevant K for that company (operator or cluster)
- K_HPC_central - HPC K in that month (repeated across all three company blocks)
- K_IPC_central - IPC K in that month
- K_cluster_central - HPC + IPC (same in all three blocks)

24. Save as 20_K_cluster_Haifa_HPC_IPC_monthly.tsv.

This design makes it easy to:

- Subset to **HPC** or **IPC** only.
 - Use **cluster-level K** directly in regressions (filter `company == "Haifa cluster (HPC + IPC)"`).
 - Always have the components available for decompositions and robustness checks.
-

6. Key choices, QA, and implications for the thesis

6.1 Units, real vs nominal, and comparability

- All series are in **thousands of NIS**, deflated using the **same OECD capital deflator** rebased to 2019.
- This ensures that:
- HPC and IPC K can be **added** meaningfully to obtain cluster K.
- Future SIPG K series (once you obtain Bayport financials) can be deflated with the same index and stacked without unit issues.

6.2 Depreciation structure and scenarios

- You allow for three depreciation scenarios (4%, 6%, 8%), but in the current scripts you consistently treat **as the baseline**.
- Path B is constructed using the **central PIM scenario**; big projects and background K are calibrated so that December Path B stocks are close to the central PIM stocks.
- In principle you can construct alternative Path B series for other δ values by re-running Step 2 and Step 4.

6.3 Treatment of intangibles

- The PIM scripts have a toggle to include real intangible investments (e.g. software) in .
- You currently leave this **off** for both HPC and IPC, reflecting the view that **Track B should focus on physical capital** relevant to port capacity.
- Intangibles (if studied later) can be incorporated either via a separate Track or by re-running the pipeline with the toggle switched on.

6.4 Early 2018 and sample start

- You explicitly **drop Jan–Nov 2018** from the cluster K panel, because:
- HPC has no pre-2018-12 monthly Path B information; December 2018 is the first anchor.
- IPC's 2018 months before December are purely interpolated from the December 2018 stock and add little informational content.
- This aligns the **K panel** with your **throughput and labor panels**, which also effectively start in late 2018.

6.5 Shape of K over time

- HPC's Path B series shows a **gradual decline** in operator-owned K over 2018–2024, reflecting asset sales (e.g. the marine department) and limited new capex.
- IPC's PIM-lin series for Haifa shows a **large step-up in K around 2018–2019** (Bayport-related infrastructure) and then relatively flat but slightly declining stocks as depreciation dominates smaller ongoing investment.
- The **cluster-level K** (HPC + IPC) is therefore **roughly flat to mildly declining** over 2019–2024:
- IPC's heavy infra investments offset HPC's disinvestment initially.
- Over time, as those assets start depreciating and Bayport is completed, net cluster K inches down in real terms.

- Once SIPG financials are incorporated, you expect **cluster K to rise more clearly** in the years around Bayport opening, because SIPG's own cranes and equipment are substantial.

6.6 Implications for K/L and the event-study design

- Since K is **roughly flat or slightly declining** while labor inputs are expected to **fall** with reform (especially at HPC), the implied **K/L ratio will tend to rise**.
- This is conceptually natural for your narrative:
- The reforms reallocate traffic and capital between operators but do not immediately trigger a doubling of aggregate infra; instead, they increase **capital intensity per worker** and **utilization**.
- For your **mediation analysis**, this makes K a plausible channel:
- At the **operator level**, HPC's K declines with privatization, but you can show that HPC's **labor cuts are more pronounced**, so HPC's K/L may still increase.
- At the **cluster level**, cluster K/L increases more clearly once SIPG K is added.

6.7 QA and robustness

The pipeline builds in several QA hooks:

- Step 2 (annual PIM) prints **book vs PIM gaps** and flags years where flows are imputed.
- Step 3 (monthly PIM-lin) carries these flags into the monthly panel so you can drop or downweight imputed years.
- Path B (HPC) constructs a **QA table** at annual anchors comparing Path B stocks to central PIM, ensuring lumpy modeling does not drift from the accounting backbone.
- The cluster merge is deliberately simple: it only sums HPC and IPC series after harmonizing units and dropping suspicious early months.

These hooks allow you to justify the K-series construction in the thesis text and to design robustness checks (e.g. using PIM-lin instead of Path B, changing δ , excluding imputed years).

7. Remaining extensions and to-dos

1. SIPG (Haifa Bayport) K series

2. Once you obtain SIPG/Bayport financial data, you can:

- Build a SIPG Step 1 → Step 2 → Step 3 pipeline, mirroring IPC.
- Optionally construct a Path B if you can identify big Bayport projects and commissioning dates.
- Extend the cluster script to include SIPG in the sum: $K_{cluster} = K_{HPC} + K_{IPC} + K_{SIPG}$.

3. Ashdod cluster

4. Repeat the entire architecture for **APC** and **TIL** plus IPC's Ashdod assets.

5. Alternative depreciation scenarios

6. Re-run the PIM and Path B scripts under different δ choices (e.g. low and high scenarios) to produce alternative K series and check sensitivity.

7. Integration with labor and throughput panels

8. When monthly labor-hours arrive, join the K panel by `(port, company, month)` to construct `ln(K/L)` mediators and capacity controls for the event-study regressions.

9. Documentation and replication

10. Keep this report in the repo alongside the code.
11. Add short README files in `Thesis/Data/K/` pointing to this report and describing how to rerun the pipeline end-to-end.

This completes the summary of how you constructed K-series for Haifa's legacy operator and landlord (HPC and IPC), and how you combined them into a Haifa-cluster K series suitable for your Econ H191 thesis econometric work.